JavaScript Lecture Notes

Callbacks & Callback Hell • Promises • async / await • DOM Manipulation • Forms & Validation

Lecture date: 25 October 2026 | Instructor: Abhishek Purohit

> **Extra (not said in the lecture):** These notes follow the order in which the lecture was taught. Boxes labelled **Instructor's point** are the instructor's own explanations. Boxes labelled **Extra** are my additions for clarity or revision and were **not** said in the lecture. The instructor's earlier-lecture code for sum/factorial/square was not in the transcript, so where I show it, I reconstructed it from the behaviour he described (marked clearly).

Contents

Part	Topics
A	Recap of async + setTimeout; why callbacks exist; why they are called 'callback'; async functions you cannot change; dependent tasks; nested callbacks; callback hell
B	Promises: real-life analogy, states, creating (executor, resolve, reject), then/catch, passing data, returning promises from functions, promise-based sum/factorial/square, chaining
C	async / await: await rules, async keyword, try/catch for rejections, when to use then/catch vs await
D	DOM: what it is, DOM tree, document object, DOM manipulation, getElementById, innerHTML, onclick, style, remove, createElement/appendChild, setAttribute/getAttribute, show/hide password, bulb task
E	Forms: reading input with .value, calculator + parseInt, onsubmit/action, required-field validation, error messages, keyup events, reusable validation, next-session preview
F	Revision: summary, definitions, syntax, comparison tables, mistakes, interview Q&A, exercises, checklist, source-coverage check

PART A - Callbacks and Callback Hell

1. Recap: asynchronous execution and setTimeout

In the last session we studied how asynchronous code is executed inside the JS engine. To **simulate** a time-consuming operation we used the `setTimeout` function.

- `setTimeout` delays the execution of a function by the given number of milliseconds. It lets us pretend that a process takes 1, 2 or 3 seconds.
- We used it because, at this point, we do not have any real operation that naturally takes that much time.
- `setTimeout` works **asynchronously**: while its timer is running, the rest of the statements are **not** made to wait. They continue executing, and meanwhile the timer also runs.

2. Why do we need callback functions?

We already knew *what* a callback function is and *how* to use one. Today's question is **why** JavaScript introduced them.

> **Instructor's point:** The core purpose of callback functions is to make a process asynchronous. Without a callback function, asynchronous execution is not possible. Whenever there is an asynchronous process, there will always be a callback function.

`setTimeout(function () { ... }, 1000)` is only *one* example. Later, when you build a back end in Node.js (for example with Express) or a front end in React or Angular, you will meet many predefined library functions that are **asynchronous by default**. Each of them will ask you for a callback.

3. Why is it called a 'callback'?

Think about the function you pass to `setTimeout`. Is it called immediately, or at some later point of time? **Later.** That is the whole idea behind the name.

- A callback function is **never executed immediately**.
- It waits for some **event** to occur and then is 'called back' into the program.
- In `setTimeout`, the event is *'timer completed'*. Other possible events: the server has sent a response, the database read has finished, etc.
- The function is handed to the JS engine (it goes into the call stack machinery) but it has **not executed yet**. Only when the event happens is it called back.

> **Instructor's point:** Definition used in class: a callback function is a function that does not get called immediately; it is executed later, after some event has been fired.

4. Async functions provided by libraries - you cannot change their nature

When you write your own sum or factorial logic, it is in the developer's hands to run it synchronously. But when a library (Node.js, React, Angular...) gives you a function that runs asynchronously, you **cannot** change it to synchronous. You cannot e-mail Node.js and ask for a 'synchronous version'.

The instructor used function names to make the difference clear (these are imaginary example functions):

Function (example)	Nature	Meaning
`sendResponse()`	Synchronous	Rest of the statements **wait** until the response is sent (10 s? they wait 10 s).
`sendResponseAsync()`	Asynchronous	Rest of the statements (S1, S2, S3...) continue immediately while the response is being sent.
`readDataFromDB()` / `writeDataInDB()`	Synchronous	Next statements wait for the read / write (insert / update query) to finish.
`readDataFromDBAsync()` / `writeDataInDBAsync()`	Asynchronous	Next statements do not wait; the operation runs 'meanwhile'.

> **Instructor's point:** By reading a function's name you can often guess its behaviour. 'Async' in the name means it will run asynchronously and will give you a callback.

5. The problem: dependent tasks with async functions

5.1 Task 1: read data from DB and send it in the response

Suppose the only functions available are `readDataFromDBAsync` (asynchronous) and `sendResponse`. If you write them one after another at the same level:

```
readDataFromDBAsync();     // starts reading, does NOT block
sendResponse();            // runs immediately - before data arrives!
```

The query might take 2-3 seconds, but `sendResponse()` does not wait. The response goes out before the data exists, so this approach **fails** the task. If the data arrives later, nothing can be done.

5.2 The solution: use the callback the async function gives you

Every async function gives you a callback. You pass a function; it is called **internally** once the data has been read. Code after the call keeps running meanwhile. Put the dependent work inside the callback:

```
readDataFromDBAsync(function () {
    // runs only AFTER data from the DB has arrived
    sendResponse();
});
// statements written here run without waiting
```

5.3 Task 2: read from one table and write into another table

Here the *job* is synchronous in nature (write only after read finishes) but the *available functions* are asynchronous. This is the problem statement to understand:

> **Instructor's point:** Job nature = synchronous (read, then write). Function nature = asynchronous (cannot be changed). So we must arrange / work around the functions to get synchronous behaviour.

If the two operations were independent (read from any table, unrelated write) you could let them run together. Here they are **dependent**.

Can we say a statement `stmt` written after `readDataFromDBAsync(...)` will run after exactly 2 seconds (if the read takes 2 s)? **No.** The only thing guaranteed is that the *callback* runs after the read completes. Likewise the write's callback runs only after the write completes.

5.4 Nesting the callbacks

Where do we call the write function? **Inside the callback of the read function.** Why? If you place both calls side by side they start together (async nature); the write never waits for the read.

```
readDataFromDBAsync(function () {
    // read is finished here
    writeDataInDBAsync(function () {
        // write is finished here
        sendResponse();          // correct place for the final response
    });
});
s1; s2; s3;                      // these are not blocked and run immediately
```

- Control first goes to the read function. Statements after it (s1, s2, s3) continue; the read's callback waits (e.g. 2 seconds).
- When the data arrives, the read callback runs and calls the write function. Again the following statements continue but the write callback waits.
- When the write completes, its callback runs, and only **then** `sendResponse()` is called.
- If `sendResponse()` had been written directly after the read call, it would execute directly (not wait).

> **Instructor's point:** Putting one callback inside another is called **nesting of callback functions**. Nesting is needed only when async functions must run in sequence (synchronously). If you want them asynchronous, do nothing extra.

> **Instructor's point:** A student asked: if write is synchronous, do we still nest? Answer: no. A synchronous function gives no callback, so you simply call `sendResponse()` on the next line. You nest only for async functions. If the write is async but another operation depends on it, that goes inside its callback.

6. Phase 1 demo: dependent sum -> factorial -> square using nested setTimeout

Since we have no real read/write functions, we use three `setTimeout`s as three asynchronous operations (sum: 1.5 s, factorial: 3 s, square: 2 s).

6.1 The earlier independent version (all at the same level)

Earlier requirement: sum of any two numbers, factorial of any number, square of any number - independent. Put at the same level, all three timers start **together**, and the one with the smallest delay finishes first.

New requirement: calculate the **sum**; then the **factorial of that sum**; then the **square of that factorial**. We still cannot make `setTimeout` synchronous, so we nest.

6.2 Code (reconstructed from the lecture; x = 3, y = 2)

```
console.log("Started calculating sum");

setTimeout(function () {                        // 1st async operation: sum (1.5 s)
    let x = 3, y = 2;
    let z = x + y;
    console.log("Sum is " + z);

    console.log("Started calculating factorial");
    setTimeout(function () {                    // 2nd async operation: factorial (3 s)
        let fact = 1;
        for (let i = 1; i <= z; i++) {          // factorial of the SUM (z), not of n
            fact = fact * i;
        }
        console.log("Factorial is " + fact);

        console.log("Started calculating square");
        setTimeout(function () {                // 3rd async operation: square (2 s)
            console.log("Square is " + (fact * fact));
        }, 2000);

    }, 3000);
}, 1500);
```

Output

```
Started calculating sum
        ... 1.5 s ...
Sum is 5
Started calculating factorial
        ... 3 s ...
Factorial is 120
Started calculating square
        ... 2 s ...
Square is 14400
```

6.3 Step-by-step explanation

1. `Started calculating sum` is logged at once; the first timer (1500 ms) starts. Even factorial has not started yet.
2. After 1.5 s the first callback runs: it computes the sum, logs it, then logs 'Started calculating factorial' and starts the **second** timer (3 s). That timer starts only *after* the sum is done.
3. The factorial must be of the sum, so the variable `z` is used (that was the new requirement).
4. The square logic is moved inside the factorial callback, because it depends on the factorial result.

6.4 Timing (students asked why it takes longer)

Operation	Waits before it STARTS	Own time	Result appears at
Sum	0 s	1.5 s	1.5 s
Factorial	1.5 s	3 s	4.5 s
Square	4.5 s	2 s	6.5 s

> **Instructor's point:** It is slower than the independent version only because the operations are dependent. This is not a bad approach; it is the requirement. Like real life: food must be cooked first, then eaten - you cannot 'save time' by eating while it cooks. If operations are independent and you serialise them anyway, that is a blunder; if they are dependent, the waiting is natural.

7. Callback Hell

This was only 3 tiny operations. In a real application you may have 10-15 dependent operations, each nested inside the previous callback, so the code keeps opening up to the right and forms a **triangle**.

> **Instructor's point:** Nesting of callback functions is called **callback hell**. It hurts readability, maintainability, debugging and scalability. You should always avoid it. The goal (multiple async functions executed synchronously) does not change; only the *way of achieving* it changes.

Problem with nesting	Why it matters
Hard to read	Each dependent step is one level deeper (triangle shape).
Hard to maintain	Complexity grows with every new feature.
Hard to debug and scale	If it is not easy to read or maintain, it is not easy to debug or extend.

To avoid callback hell we use **Promises**.

PART B - Promises

8. The real-life idea behind a 'promise'

The JavaScript mechanism is named after (and works exactly like) a real-life promise.

Example from class: the instructor promises the class a treat at the Taj Hotel next Saturday.

- Today (promise made) we **cannot say** whether it will be fulfilled. It is **pending**.
- On Saturday, if he turns up, the promise is **fulfilled**; if he does not, it is **rejected**.
- Based on the outcome you take different actions: promise kept -> order food (happy flow); promise broken -> a different set of actions (go looking for him!).
- There is no guarantee any promise will be kept - the same is true in JavaScript.

In JavaScript: for any operation or task (fetch data from a server, calculate factorial, LCM, HCF...) you **create a promise**. If the logic completes successfully the promise is **fulfilled / resolved**; if there is an error (server error, code error, anything) it is **rejected**. Based on the final state you decide what to do next (use the result, retry, log an error, handle the error...).

9. Promise lifecycle and states

1. **Create** a promise for the operation / logic.
2. Until the logic completes the promise stays in the **pending** state. You cannot say anything about its outcome yet.
3. After the operation executes: successful -> **fulfilled (resolved)**; failed -> **rejected**.
4. Based on the state, take actions: one set of logic for fulfilment, another for rejection (a bit like if-else, but not exactly if-else).

State	Meaning	Real-life example
Pending	Created; operation not finished. Initial state.	Promise made, Saturday not arrived.
Fulfilled / Resolved	Operation completed successfully.	Instructor shows up at the Taj.
Rejected	Error or failure occurred.	Instructor does not show up.

> **Instructor's point:** The promise never fulfils or rejects **by itself**. The **developer** decides, based on the logic written, when to resolve and when to reject. The JS engine only executes. If you never resolve or reject, the promise stays **pending forever**.

10. Creating a promise - syntax

A promise is an object of the `Promise` class, created with `new`, just like `new Student()` or `new Employee()`. You pass a **callback function** called the **promise executor function** that contains the logic for which you are making the promise.

```
let myPromise = new Promise(function (resolve, reject) {
    // all the logic for which you are making the promise goes here
    // call resolve(...) on success, reject(...) on failure
});

// Arrow function version is also fine:
let myPromise2 = new Promise((resolve, reject) => {
    // ...
});
```

10.1 The two automatic arguments

- The executor is called **internally** by the JS engine (not by you), and it automatically receives **two arguments**; you just declare them.
- Both arguments are **functions**. The **first** marks the promise as resolved; the **second** marks it as rejected.
- You may name them anything (x, y; a, b; p, q...). `resolve` **and** `reject` **are NOT keywords** - they are just meaningful names. (The instructor joked you could even name them with silly names.)
- The signature stays exactly the same for every promise; only your logic and conditions change.
- If you never call either function, the promise stays pending forever.

10.2 Where do we call resolve/reject?

- Call it at the point where your logic considers the operation *complete* - often the last line (e.g. after sending a request, waiting for the response and processing 10-15 lines of code).
- Calling `resolve`/`reject` only changes the promise's internal state. The following code is not 'stopped'; but you then act according to the state.
- Do not call resolve/reject inside a loop. A loop performs calculation/iterations; once the iterations or logic are finished, you decide success or failure.

11. Demo: 'every number is even' promise

The promise: *'I promise the number is always even.'* This is just to learn the syntax - not a real-world use (an `if/else` could do this).

```
<html>
<head></head>
<body>
<script>
    let n = 4;                                  // try 7 as well
    let myPromise = new Promise(function (resolve, reject) {
        if (n % 2 === 0) {
            resolve();                          // fulfilled
        } else {
            reject();                           // rejected
        }
    });
    console.log(myPromise);                     // inspect the state
</script>
</body>
</html>
```

- When is the promise object created? **Immediately** when the script tag loads - it is not waiting for a button click or user input.
- At that same moment the **executor runs**. So the promise is settled as soon as the page loads.
- A promise object itself is created and its executor runs **synchronously**; only operations inside the executor (like `setTimeout`) can be asynchronous.

Output

```
n = 4  ->  Promise {<fulfilled>: undefined}
n = 7  ->  Promise {<rejected>: undefined}
```

12. Handling a promise: then() and catch()

To react to the state, JS gives two functions, called on the promise object and chained:

```
myPromise
    .then(function () {
        console.log("Promise fulfilled, number is even");
    })
    .catch(function () {
        console.log("Promise rejected, number is not even");
    });
```

Function	Runs when	You pass
`then(callback)`	The promise is resolved / fulfilled	A callback with the success logic
`catch(callback)`	The promise is rejected	A callback with the failure logic

- For `n = 4` the promise resolves, `then` runs and prints 'promise fulfilled and number is even'.
- For `n = 7` the promise is rejected, `catch` runs and prints 'promise rejected, number is not even'.
- Full flow: create promise -> promise executes -> state changes -> handle with then/catch.

13. Passing data from resolve/reject to then/catch

Sometimes data inside the executor (a number, string, boolean, array, object, even a function) is needed in `then` or `catch`. Pass it as the argument of `resolve` / `reject`; it is received as the parameter of the callback.

```
let n = 7;
let myPromise = new Promise(function (resolve, reject) {
    if (n % 2 === 0) {
        resolve("Promise fulfilled, number is even");   // goes to then
    } else {
        reject("Promise rejected, number is not even"); // goes to catch
    }
});

myPromise
    .then(function (arg) {            // arg  <- argument passed to resolve
        console.log(arg);
    })
    .catch(function (errorMsg) {      // errorMsg <- argument passed to reject
        console.log(errorMsg);
    });
```

Output

```
n = 7  ->  Promise rejected, number is not even
n = 6  ->  Promise fulfilled, number is even
```

> **Remember:** `resolve(x)` -> received by `then`. `reject(x)` -> received by `catch`. The parameter names (arg, errorMsg) are your choice.

14. Returning a promise from a function

In real projects the promise is created inside a function, so the logic can be reused with different inputs.

```
function checkEven(num) {
    return new Promise(function (resolve, reject) {
        if (num % 2 === 0) resolve("even");
        else reject("not even");
    });
}

// What does checkEven(4) return?  Not a string - it returns a PROMISE OBJECT.
let rs = checkEven(4);        // rs is a promise object
rs.then(...).catch(...);

// Simpler: chain directly on the call
checkEven(4)
    .then(function (msg) { console.log(msg); })
    .catch(function (err) { console.log(err); });
```

Minor tweak shown: instead of `let p = new Promise(...); return p;` you can simply `return new Promise(...)`. The result is identical.

15. Phase 3: Preventing callback hell with promises (sum, factorial, square)

For each operation we create a separate function that returns a promise. **Important:** such a function does **not** return the number (e.g. 5) - it returns a promise object. You get the value by handling the promise with `then`/`catch`.

15.1 The sum function

```
function sum(a, b) {
    console.log("Started calculating sum");
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            if (a > 100 || b > 100) {                    // fake scenario: upper limit
                reject("Upper limit has been crossed. Values should be less than 100.");
            } else {
                let c = a + b;
                resolve(c);                              // c is passed to then
            }
        }, 1500);                                        // simulate 1.5 s of work
    });
}

sum(2, 3)
    .then(function (sumResult) { console.log("Sum is " + sumResult); })
    .catch(function (error)    { console.log(error); });
```

15.2 Dry run for sum(2, 3)

1. `sum(2, 3)` runs: logs 'Started calculating sum'.
2. A promise object is created; its executor starts `setTimeout` (1.5 s). The function returns the promise (pending).
3. After 1.5 s the callback checks the condition (2 > 100 or 3 > 100? No), computes `c = 5` and calls `resolve(5)`.
4. Because the promise resolved, `then` runs with `sumResult = 5`.

Output

```
Started calculating sum
        ... 1.5 s ...
Sum is 5
```

With `sum(200, 3)` the condition is true, the promise is **rejected** and `catch` prints the error message.

Output

```
Started calculating sum
        ... 1.5 s ...
Upper limit has been crossed. Values should be less than 100.
```

> **Instructor's point:** The condition uses OR (||). If **either** a or b is above 100 the operation fails (a student asked about both being above 100 - still fails).

15.3 The factorial function (same structure)

```
function factorial(num) {
    console.log("Started calculating factorial");
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            let result = 1;
            for (let i = 1; i <= num; i++) {
                result = result * i;
            }
            resolve(result);          // you may add a reject condition (e.g. upper limit)
        }, 3000);                     // 3 seconds
    });
}
```

The lecture left rejection for factorial as an exercise. If you never use `reject`, a `catch` attached to it will simply never run - and not passing `resolve`/`reject` as parameters is also legal.

> **Extra (not said in the lecture):** Likewise a `square(num)` function with a 2-second timer returning a promise is assumed (the instructor referred to it as 'if we assume a square function'). I reconstructed it as: `return new Promise(r => setTimeout(() => r(num * num), 2000));`

16. Chaining promises with then().then()

16.1 Wrong approach - nesting then inside then (back to callback hell)

```
sum(2, 3).then(function (sumResult) {
    let rs = factorial(sumResult);        // rs is a PROMISE OBJECT, not a number
    rs.then(function (factResult) {       // handling it inside -> nesting again!
        console.log("Factorial is " + factResult);
    });
});
```

Students noticed this is nesting again. 'We avoided gold-plated callbacks and still asked for paint' - that is not what we want.

16.2 Correct approach - return the promise from then

The value returned from a `then` callback is passed on, and if it is a promise, the **next** `then` handles it. So **return** the factorial promise and attach another `then`:

```
sum(2, 3)
    .then(function (sumResult) {
        console.log("Sum is " + sumResult);
        return factorial(sumResult);          // returns a promise
    })
    .then(function (factResult) {             // handler for the factorial promise
        console.log("Factorial is " + factResult);
        return square(factResult);            // returns another promise
    })
    .then(function (sqResult) {               // handler for the square promise
        console.log("Square is " + sqResult);
    })
    .catch(function (error) {                 // ONE catch for the entire chain
        console.log(error);
    });
```

Output

```
Started calculating sum
Sum is 5                  (after 1.5 s)
Started calculating factorial
Factorial is 120          (after 3 s more = 4.5 s total)
Started calculating square
Square is 14400           (after 2 s more = 6.5 s total)
```

- Instead of nesting, we are now **chaining**. Operation 2 starts only after operation 1's promise resolves; operation 3 only after operation 2's.
- Each `then` is the handler for the promise returned by the previous step; its parameter is the value that step resolved with.
- **One** `catch` is enough for all: only one promise can reject in the chain. If the first promise rejects, no later `then` runs and control goes to the last `catch`. If the second rejects, also this same `catch`.
- The original nature of sum and factorial is still asynchronous; we made them dependent using then/catch and promises.

16.3 What is still wrong?

We avoided nesting but are now stuck in **long chains** of `then`. For many operations that is also tiresome. The instructor said chaining is okay for small implementations, and that there is one more approach: **async and await**.

PART C - async and await

17. The await keyword

Instead of calling `.then()` you can write `await` in front of a function call that returns a promise.

```
let rs = await sum(2, 3);       // rs = 5
```

`await` does **two jobs**:

1. It makes the **rest of the statements wait** until the promise of that function is resolved (i.e. the function finishes).
2. When the promise resolves, it **returns the resolved value** (the argument you passed to `resolve`), which you can store in a variable.

> **Instructor's point:** Without `await`, `sum()` returns a promise object, and `rs` would just be that promise, not 5. The rest of the statements also would not wait.

> **Remember:** `await` returns only the **resolved** value. It does not return rejected values. Rejections need try/catch.

You can use `await` before **any** function call that returns a promise: `await factorial(sumResult)`, `await square(factResult)`. Compare this to the then-callback-return-chain: it is far simpler to read.

18. await only works inside an async function

- `await` cannot be written directly at the top level or outside a function; it must be inside a function.
- That function's signature must be marked with the `async` keyword, otherwise you get the error: `Await is only valid in async functions` (the instructor demonstrated this error).
- `async` on the function signature declares that the function deals with an asynchronous process.
- Functions never call themselves, so you must also call the function (`handlePromise()`).

19. Handling rejection with try/catch

There is no separate keyword to handle rejection with `await`. Put the statements where rejection is possible inside `try`; if any awaited promise rejects, control jumps to `catch`, and the rest of the try block does not execute. The `catch` parameter receives what was passed to `reject`.

```
async function handlePromise() {
    try {
        let sumResult = await sum(2, 3);
        console.log("Sum is " + sumResult);

        let factResult = await factorial(sumResult);
        console.log("Factorial is " + factResult);

        let sqResult = await square(factResult);
        console.log("Square is " + sqResult);
    } catch (error) {
        console.log(error);          // rejected value from any step
    }
}

handlePromise();                     // call the function!
```

Output

```
Started calculating sum
Sum is 5
Started calculating factorial
Factorial is 120
Started calculating square
Square is 14400
```

- If sum's promise rejects, the factorial and square lines never run and the `catch` block runs.
- If sum resolves but factorial rejects, `Sum is 5` is printed, then control goes to `catch`.
- try/catch is already known from Java; in JS it can handle ordinary exceptions and also promise rejections.

> **Instructor's point:** A student asked: can we use await inside `.catch()` or `.then()` callbacks? Technically yes, but nobody does because async/await with try/catch is more readable.

20. The async keyword - second use (documentation)

Besides being required where `await` is used, you can put `async` on a function that **returns a promise** (even with no await inside). Nothing changes in behaviour; it is a declaration so other developers can understand from the signature that the function returns a promise. Good developers do this in real projects.

21. Scope of await, and then/catch vs await

- `await` makes only the code **inside that function** wait. Code written after the call to `handlePromise()` outside the function does **not** wait - the function as a whole still runs asynchronously relative to other code.
- Use **await** when you have dependent operations that must run in sequence (synchronously).
- Use **then/catch** when a function is asynchronous and you want it to **stay asynchronous**; you must not use await there.

Approach	When used	How errors are handled	Readability
Nested callbacks	First technique for dependent async work	Manually in each callback	Poor (callback hell)
Promise + then/catch (chaining)	Handle a promise; chain dependent steps	`catch` at end of chain	Better, but long chains
async / await + try/catch	Dependent async steps that need sequential order	`try / catch` block	Best, looks synchronous

Progression summary

Callbacks (needed to do async) -> nesting callbacks for dependent steps -> callback hell -> Promises with then/catch -> chains of then -> async/await. The *goal* (run async functions synchronously) never changed; only the technique improved.

PART D - DOM Manipulation

22. What is the DOM?

DOM = Document Object Model. When you open an HTML page, the browser does not directly display the tags. First it **reads the HTML code and prepares a tree-like structure of all the tags** in its internal memory. That tree is the DOM.

Example page: html -> head (title) and body (h1, div containing p, form containing input).

```
html                      (root)
 |-- head
 |    `-- title
 `-- body
      |-- h1
      |-- div
      |    `-- p            (p is child of div)
      `-- form
           `-- input        (input is child of form)
```

- Using this tree the browser knows which element is a **parent**, **child** or **sibling**, and how to render each.
- The browser does not re-read the source code again and again; it consults the DOM.
- 'Model' = a virtual representation. The DOM represents the hierarchy of all tags in the document.
- If you have thousands of tags, the browser builds a tree for all of them.

Questions answered in class

Question	Instructor's answer
How does external CSS get loaded?	The `link` tag becomes a child of `head` in the tree; the browser reads its `href` attribute and loads the CSS file.
Internal / inline CSS?	A `style` tag is a child of `head`; the browser reads it, stores the CSS in memory and applies it when it meets matching elements. For inline CSS, the `style` attribute is applied when that element is read in the body.
If HTML changes, is the tree rebuilt?	Yes, if you change the source code and reload, the browser prepares the structure again.
What about caching?	Browsers cache some things (styles, data). If your changes do not appear, do a hard reload / clear the cache.

23. The document object

Earlier we described `document` as an object representing the whole web page. In reality it is the **object of the DOM**: the object that represents / holds the tree structure the browser built. To perform any operation on the DOM you use the `document` object.

24. What is DOM manipulation?

Changing the HTML source and reloading is **not** DOM manipulation. DOM manipulation means that after the page has loaded and the DOM is prepared, you **dynamically**, at run time, change that tree using JavaScript - without reloading the page and without editing the source code.

Examples: the user clicks a button and an element is removed; a new element appears; an error message appears when the user types an invalid value in a field.

What you can do with DOM manipulation

1. Create a new element dynamically (insert it into the DOM).
2. Remove an existing element.
3. Change the content of an element (e.g. h1 'Hello World' -> 'Bye World').
4. Get (read) the content of an element, including what is typed in an input.
5. Change styles dynamically (e.g. green border for valid input, red border for invalid).
6. Change attribute values (e.g. `src` of an img, `href` of an anchor).
7. Get (read) attribute values.

25. The approach: first get the object

In JavaScript everything is an object; each HTML tag is also an object in the DOM. To do *anything* to an element (change content, style, remove...) you must first **get its object**. On that object you then use predefined properties and functions.

26. document.getElementById()

You give the unique `id` of an element; the function searches the DOM tree for a match and returns the **element object (reference)** - not just its text.

```
<p id="para">This is para demo</p>

<script>
    let obj = document.getElementById("para");
    console.log(obj);              // prints the whole <p> element, not only the text
</script>
```

Instructor's point: `obj` refers to the entire paragraph tag, not only the content string. If no element has that ID, the result is `undefined`/null-like and using `.style` on it will throw an error. The instructor mentioned optional chaining as a way to avoid that error.

Extra (not said in the lecture): Strictly, `getElementById` returns `null` (not `undefined`) when nothing matches. Optional chaining looks like `obj?.style` - not covered in detail in the lecture.

27. Reading content: innerHTML

```
let obj = document.getElementById("para");
console.log(obj.innerHTML);        // This is para demo
```

`innerHTML` gives the content inside the element (paragraph, h1, div, etc.). Console output: `This is para demo`.

28. Running code on a button click: onclick

Earlier the code ran at page load. We want it to run only when the user clicks a button. Put the code in a function and call that function from the button's `onclick` attribute.

```
<p id="para">This is para demo</p>
<button onclick="getContent()">Get Content</button>

<script>
    function getContent() {
        let obj = document.getElementById("para");
        console.log(obj.innerHTML);
    }
</script>
```

Instructor's point: Functions do not call themselves. If you do not call `getContent()`, nothing will be logged. The `onclick` event fires when the user clicks; it calls the function and control jumps into it.

29. Changing content: innerHTML assignment

Getting the element object is fixed whether you read or change content. To change content, assign a value to `innerHTML`.

```
<p id="para">This is para demo</p>
<button onclick="changeContent()">Change Content</button>

<script>
    function changeContent() {
        let obj = document.getElementById("para");
        obj.innerHTML = "updated content";
    }
</script>
```

Output

```
Before click:  This is para demo
After click:   updated content
```

In the browser's **Elements** tab you can watch the DOM change dynamically when you click, while the source file is unchanged. A reload brings back the original content (because the DOM is rebuilt from the source).

30. Changing style: element.style

The `style` property holds all CSS properties. CSS names with hyphens become **camelCase** in JS (no hyphen, next word starts with an uppercase letter).

CSS property	JavaScript property
color	`obj.style.color`
background-color	`obj.style.backgroundColor`
font-size	`obj.style.fontSize`
border-bottom-color	`obj.style.borderBottomColor`
padding-top / padding-left	`obj.style.paddingTop` / `obj.style.paddingLeft`

```
<style>
    #para { background-color: red; color: white; }     /* initial style */
</style>

<p id="para">This is para demo</p>
<button onclick="changeStyle()">Change Style</button>

<script>
    function changeStyle() {
        let obj = document.getElementById("para");
        obj.style.color = "red";
        obj.style.backgroundColor = "white";
    }
</script>
```

Result: initially red background with white text; after the click, red text on a white background. **Execution flow:** button click -> `onclick` calls `changeStyle()` -> `getElementById` finds the element in the DOM -> its style properties are updated -> the page shows the new styling.

31. Removing an element: remove()

```
<p id="para">This is para demo</p>
<button onclick="removePara()">Remove Para</button>

<script>
    function removePara() {
        let obj = document.getElementById("para");
        obj.remove();                  // deletes the element from the DOM
    }
</script>
```

After clicking, the `p` element disappears from the page and from the Elements tab. After a reload it returns because the source code still contains it.

32. Creating a new element: createElement() and appendChild()

Task: when the user clicks a button, add a new heading inside an empty `div`.

1. `document.createElement("h1")` creates a new HTML element. It exists only in **browser memory**.
2. A new element does not appear automatically. The developer must decide where it fits in the tree (whose child, whose parent).
3. Get the parent element (the `div`, given `id="container"`) with `getElementById`.
4. Call `parent.appendChild(child)` to attach it as a child.
5. A new element has no content; set it using `innerHTML`.

```
<div id="container"></div>
<button onclick="generateHeading()">Generate Heading</button>

<script>
    function generateHeading() {
        let h1Obj = document.createElement("h1");        // 1. create (in memory)
        h1Obj.innerHTML = "This is a dynamic heading";   // 2. give it content
        let container = document.getElementById("container");
        container.appendChild(h1Obj);                    // 3. put it inside the div
    }
</script>
```

Before the click the div is empty. After the click, expanding the div in the Elements tab shows a new `h1` child.

> **Common mistake:** If you only call `createElement` and skip `appendChild`, nothing appears on the page - the instructor showed this as proof.

33. Working with attributes: setAttribute() and getAttribute()

Used when you need to change something that is an **attribute** (like input `type`, `src`, `href`) rather than content.

- `element.setAttribute(attributeName, newValue)` - takes two arguments.
- `element.getAttribute(attributeName)` - returns the value of that attribute.
- Use **innerHTML** for content of tags like h1/p/div. Use **setAttribute** for attributes such as the `type` of an input.

33.1 Show / hide password (toggle)

A password field hides what you type. To show it, change its `type` attribute from `password` to `text`. The button text also changes ('Show' <-> 'Hide'), so a condition decides which action to perform. The function is named `togglePassword` rather than `showPassword`.

```
<input type="password" id="password-field" placeholder="Enter password">
<button id="btn" onclick="togglePassword()">Show</button>

<script>
    function togglePassword() {
        let passwordField = document.getElementById("password-field");
        let button = document.getElementById("btn");

        if (button.innerHTML === "Show") {
            passwordField.setAttribute("type", "text");      // reveal
            button.innerHTML = "Hide";
        } else {
            passwordField.setAttribute("type", "password");  // hide again
            button.innerHTML = "Show";
        }
    }
</script>
```

- First click: the button reads 'Show' so the type becomes `text` and the button changes to 'Hide'.
- Second click: the button reads 'Hide' so the type goes back to `password` and the button reads 'Show'.
- Without the `if/else` the button would only change once; the toggle needs a condition (this is what the instructor showed step by step).

33.2 Task discussed: bulb on/off

Build a page with a bulb and a button to turn it on/off. The approach agreed in class: keep two images (bulb on, bulb off) and on each click change the `src` attribute of the image using `setAttribute` (check the current value with `getAttribute` if you want to toggle). A student also suggested changing the background colour (yellow when on, white when off) - also fine. No full code was given; it was left as an exercise (see revision).

PART E - Forms, Input and Validation

34. Reading user input: the value property

The DOM can read form data and validate it. For an `input` the typed text is read with `.value`, not `innerHTML`. (`innerHTML` is for tags whose content is static and not typed by the user, like h1, p, div.)

```
<input type="text" id="name-field" placeholder="Enter the name">
<button onclick="getName()">Submit Name</button>

<script>
    function getName() {
        let nameField = document.getElementById("name-field");
        console.log(nameField.value);        // what the user typed
    }
</script>
```

- `nameField` holds the entire input tag (object reference), not just the text.
- `nameField.value` is the typed text. You can then submit it to a server or process it.
- The instructor forgot to call the function once - a good reminder: **always call the function** from `onclick`.

35. Mini project: a simple calculator (add two numbers)

```
<input type="text" placeholder="Enter first number" id="field1"> <br><br>
<input type="text" placeholder="Enter second number" id="field2"> <br><br>
<button onclick="add()">Add</button>
<h3 id="result"></h3>

<script>
    function add() {
        let n1 = document.getElementById("field1").value;
        let n2 = document.getElementById("field2").value;
        let n3 = n1 + n2;                                   // BUG: string concatenation
        document.getElementById("result").innerHTML = "Sum is " + n3;
    }
</script>
```

Output

```
Input 3 and 5  ->  Sum is 35       (not 8!)
```

> **Common mistake: The culprit is `.value`.** It always returns a **string**, even if the input contains digits. `"3" + "5"` concatenates to `"35"`. (With `*`, `/`, `-` JS converts automatically, so only `+` is a problem - the instructor's answer to a student's question.)

Fix: extract the number from the string with `parseInt` (a direct JS function; in Java it was `Integer.parseInt`, here you do not write `Integer.`).

```
function add() {
    let n1 = parseInt(document.getElementById("field1").value);
    let n2 = parseInt(document.getElementById("field2").value);
    let n3 = n1 + n2;
    document.getElementById("result").innerHTML = "Sum is " + n3;
}
```

Output

```
Input 4 and 2  ->  Sum is 6
```

36. Form validation

36.1 The form

A registration form with username, password and phone number (instructor started with username and email; the final fields used were username, password, phone). The form has an `action` and a `method`.

```
<form action="https://www.google.com" method="get" onsubmit="return validate()">
    <label>Username</label>
    <input type="text" id="username-field" placeholder="Enter username"
           onkeyup="validateUsername()">
    <span id="username-error" class="error-span"></span>
    <br><br>

    <label>Password</label>
    <input type="password" id="password-field" placeholder="Enter password"
           onkeyup="validatePassword()">
    <span id="password-error" class="error-span"></span>
    <br><br>

    <label>Phone</label>
    <input type="text" id="phone-field" placeholder="Enter phone">
    <span id="phone-error" class="error-span"></span>
    <br><br>

    <button type="submit">Register</button>
</form>
```

36.2 onsubmit and action - which runs first?

When the user clicks a submit button, **two things exist**: the form's `onsubmit` event and the `action` attribute. **First `onsubmit` runs; it decides whether `action` will run, based on the Boolean it returns.**

onsubmit returns	Result
`return true`	The form's `action` is executed (the form is submitted).
`return false`	The `action` is NOT executed; submission is blocked.
no onsubmit at all	The action always executes.

So you do not hard-code true/false. You write a custom function (`validate`) that checks the fields and returns `true` when data is valid or `false` when invalid, and you write `onsubmit="return validate()"`. Note the `return` keyword before the call.

36.3 Step 1 - required validation for username

```
function validate() {
    let username = document.getElementById("username-field").value;

    if (username.length === 0) {                        // nothing typed
        document.getElementById("username-error").innerHTML = "Username is required";
        return false;
    }
    return true;
}
```

- Without an error message the user would not know why the form does not submit, so a `span` next to the input receives the message dynamically via `innerHTML`.
- With the field empty: the message appears and `false` is returned, so the action does not run.
- With valid text: `validate()` returns `true`, `onsubmit` returns `true` and the action runs.

36.4 Step 2 - add password; why only one error showed (the return problem)

When password validation was added with the same style, only the username error appeared. Why? After the username's `return false`, the function ends - the password check never executes. A return statement stops everything after it.

Fix: use a temporary flag, assume the data is valid, set it to false for each failure, and return once at the end.

```
function validate() {
    let isValid = true;                                    // assume valid

    let username = document.getElementById("username-field").value;
    let password = document.getElementById("password-field").value;
    let phone    = document.getElementById("phone-field").value;

    if (username.length === 0) {
        document.getElementById("username-error").innerHTML = "Username is required";
        isValid = false;
    }
    if (password.length === 0) {
        document.getElementById("password-error").innerHTML = "Password is required";
        isValid = false;
    }
    if (phone.length === 0) {
        document.getElementById("phone-error").innerHTML = "Phone is required";
        isValid = false;
    }
    return isValid;                                        // single return at the end
}
```

Now all fields are validated at the same time, and all messages are displayed together.

36.5 Making error messages red

Give every error `span` the same class, and write the CSS only once:

```
<style>
    .error-span { color: red; }
</style>
```

37. Clearing the error while typing: keyboard events

Expectation: when the user starts typing, the error message should disappear; if the field is emptied again, it should reappear. This needs an event that fires as the user types: **onkeyup** (as the instructor wrote it in the code), so the check is done again for every typed character.

The instructor also mentioned the `onKeyDown` event name while explaining and said another event, **onchange**, exists with a small difference.

> **Extra (not said in the lecture):** `onkeyup` fires when a key is released; `onkeydown` fires when it is pressed (before the field's value updates); `onchange` fires only after the field loses focus with a changed value. For live validation `onkeyup` (or `oninput`) works best. The lecture named the events but did not compare them in detail. The code in the lecture uses an on-key-up handler; the instructor's spoken label 'onKeyDown' was a slip.

37.1 Proof that the event fires

```
function handleOnKeyUp() {
    console.log("user has typed");
}
// <input ... onkeyup="handleOnKeyUp()">
```

Output

```
(type a)  user has typed
(type c)  user has typed
(type v)  user has typed        <- one message for every typed character
```

37.2 Live validators

```
function validateUsername() {
    let username = document.getElementById("username-field").value;
    if (username.length === 0) {
        document.getElementById("username-error").innerHTML = "Username is required";
    } else {
        document.getElementById("username-error").innerHTML = "";   // clear message
    }
}

function validatePassword() {
    let password = document.getElementById("password-field").value;
    if (password.length === 0) {
        document.getElementById("password-error").innerHTML = "Password is required";
    } else {
        document.getElementById("password-error").innerHTML = "";
    }
}
```

- No `isValid` variable and no return are needed here: these functions only show/clear the message.
- Behaviour: click Register with empty fields -> messages appear. Type in a field -> its message disappears. Delete the last character -> the message returns.
- **The phone-field version was left as a task for students** (copy the same pattern).

37.3 One reusable function instead of three

Mukul asked whether one common function can serve all fields. **Yes.** The logic is the same; only the input field ID, the message and the error-span ID differ. So create one common function (e.g. `validateOnKeyUp`) that takes these **three arguments**, and pass different values per field when calling it from each input's event.

```
// Extra: one possible shape of the idea (not coded in the lecture)
function validateRequired(fieldId, errorId, message) {
    let value = document.getElementById(fieldId).value;
    document.getElementById(errorId).innerHTML = (value.length === 0) ? message : "";
}
// <input id="phone-field"
//   onkeyup="validateRequired('phone-field','phone-error','Phone is required')">
```

38. Next session and wrap-up

- Next session: more validations - e.g. password format: 8 to 10 characters, at least one uppercase and one lowercase letter.
- Task: complete the phone-number live validation yourself.
- Instructor's reminders: add everything learned so far to your first (portfolio) question; complete the second and third assignments if not done; more assignments / updates will be shared.

PART F - Revision Material

39. Concise summary of all topics

1. Callbacks make async possible. They are never called immediately; they run after an event (timer done, response arrived).
2. Async library functions cannot be made synchronous. For dependent tasks, nest callbacks.
3. Nested callbacks -> callback hell (hard to read, maintain, debug, scale).
4. Promise = object representing the future result of an operation. States: pending, fulfilled (resolved), rejected. The developer calls resolve/reject.

5. `new Promise((resolve, reject) => {...})`; executor runs immediately; `then` handles success, `catch` handles failure; data flows through resolve/reject arguments.
6. A function returning a promise gives a promise, not a value. Chain with `.then().then().catch()` by returning promises.
7. `async`/`await`: `await` pauses the function until resolved and returns the value; only inside `async` functions; rejections via try/catch.
8. DOM = browser's tree of the page; `document` is its object; manipulation = dynamic run-time changes.
9. Core DOM tools: getElementById, innerHTML, onclick, style, remove, createElement, appendChild, setAttribute, getAttribute, `.value`.
10. Input values are strings: use parseInt for arithmetic.
11. Form validation: `onsubmit="return validate()"`, `isValid` flag, error span, keyboard event to clear errors.

40. Important definitions

Term	Definition	Purpose / key idea
Callback function	A function passed as an argument that is not called immediately; it is called later when an event happens.	Enables asynchronous behaviour.
Nesting of callbacks	A callback inside another callback.	Run async functions in sequence.
Callback hell	Deep nesting of callbacks.	Avoid it: unreadable, unmaintainable.
Promise	Object representing the eventual success/failure of an operation.	Avoid callback hell.
Promise executor	The callback passed to `new Promise`, holding the logic.	Receives resolve and reject.
Pending / Fulfilled / Rejected	The three promise states.	Decide what action to take.
await	Makes the rest wait and returns the resolved value.	Sequential async code.
async	Marks a function that uses await (or returns a promise).	Required for await.
DOM	Document Object Model: tree of the page's tags in browser memory.	Browser renders from it.
DOM manipulation	Dynamically modifying the DOM at run time.	Interactive pages.

41. Syntax cheat sheet

Task	Syntax
Create promise	`new Promise((resolve, reject) => { ... })`
Resolve / reject with data	`resolve(value)` / `reject(errorMessage)`
Handle promise	`p.then(v => ...).catch(e => ...)`
Wait for promise	`let v = await fn();` inside `async function`
Handle rejection (await)	`try { ... } catch (e) { ... }`
Get element	`document.getElementById("id")`
Read / change content	`obj.innerHTML` / `obj.innerHTML = "text"`
Read input value	`obj.value`
Change style	`obj.style.backgroundColor = "red"`
Remove element	`obj.remove()`
Create + attach element	`document.createElement("h1")`, `parent.appendChild(child)`
Attributes	`obj.setAttribute("type", "text")` / `obj.getAttribute("type")`
String to number	`parseInt(str)`
Form validation hook	

42. Key differences between related concepts

Concept A	Concept B	Difference
Sync function	Async function	Sync blocks the rest; async lets the rest continue and gives a callback.
Callback nesting	Promise chaining	Nesting goes deeper; chaining stays flat with `.then()`.
`then`	`catch`	then = resolved; catch = rejected.
Promise chain	async/await	`then` callbacks vs `await` + try/catch; await reads like normal code.
`await` with then/catch		Use await for dependent steps; then/catch to keep async.
`innerHTML`	`.value`	innerHTML = content of non-input tags; value = what the user typed in input.
`innerHTML`	`setAttribute`	Content vs attribute (e.g. `type`, `src`).
`createElement`	`appendChild`	Create in memory vs insert in the DOM.
Editing HTML + reload	DOM manipulation	Reload rebuilds DOM; manipulation changes it at run time.
`onsubmit`	`action`	onsubmit runs first and decides, by true/false, whether action runs.

43. Common mistakes to avoid

Mistake	Why / correction
Expecting a promise-returning function to return the number	It returns a promise. Use `then` or `await`.
Never calling resolve/reject	The promise stays pending forever.
Treating `resolve`/`reject` as keywords	They are just parameter names.
Calling the dependent function at the same level	It will not wait. Put it in a callback / then / use await.
Handling inner promise with a nested `then`	Return the promise and chain the next `then`.
Using `await` outside an async function	Error: 'await is only valid in async functions'.
Expecting `await` to catch rejection	Use try/catch.
Forgetting to call the function	`onclick="fn()"` or call it yourself - functions do not run on their own.
Adding input `.value`s directly	Strings concatenate (3+5 = 35). Use `parseInt`.
Using `innerHTML` to read an input	Use `.value`.
Early `return false` in validate()	Later checks are skipped. Use an `isValid` flag.
Using hyphenated CSS names in JS	Use camelCase: `backgroundColor`.
Creating an element but not appending it	Nothing shows. Call `appendChild`.

44. Interview questions and answers

> **Extra (not said in the lecture):** These are revision questions based on the lecture's concepts; the wording of the answers is mine.

Question	Answer
What is a callback function and why is it so named?	A function passed to another function that is called later, after an event, not immediately - it is 'called back'.
Why do we need callbacks?	Asynchronous execution is not possible without them; async functions call your callback when their work finishes.
What is callback hell and how do you avoid it?	Deeply nested callbacks that hurt readability and maintainability. Avoid with promises and async/await.
What are the states of a promise?	Pending, fulfilled (resolved), rejected.
Who decides whether a promise resolves or rejects?	The developer, via the logic and the calls to resolve/reject.
What happens if neither resolve nor reject is called?	The promise stays pending forever.
When does the promise executor run?	Immediately when the promise object is created.
How do you pass data from the promise to then/catch?	As arguments to resolve and reject.
How do you chain async operations without nesting?	Return the next promise from `then` and attach another `then`; one `catch` at the end.
What does `await` do?	Pauses the rest of the async function until the promise resolves and returns the resolved value.
Where can `await` be used?	Only inside an `async` function (in this lecture's scope).
How do you handle rejection with await?	try/catch.
then/catch or await?	await for dependent steps; then/catch to keep operations asynchronous.

Question	Answer
What is the DOM?	A tree representation of the page built by the browser; `document` is its object.
innerHTML vs value?	innerHTML for content of normal tags; value for user input in form fields.
Why does 3 + 5 give 35 from inputs?	`.value` returns strings. Use `parseInt`.
How does onsubmit control the form?	Return true to allow action, false to block it.

45. Practice exercises (solutions not included)

Easy

1. Explain, in your own words, why a callback is not called immediately.
2. Create a promise that resolves if a number is greater than 10 and rejects otherwise. Print messages using `then` and `catch`.
3. Add a paragraph and a button; on click change the paragraph text and its colour.

Medium

1. Write async `add`, `double`, `negate` functions with setTimeout and chain them using then/catch.
2. Rewrite exercise 1 with `async/await` and try/catch. Add a rejection condition and observe the flow.
3. Complete the phone-field live validation (empty-check with message that clears when typing).
4. Build the bulb on/off page using `setAttribute` on the image `src`.
5. Build a calculator with add, subtract, multiply, divide buttons and show the result in an `h3`.

Challenging

1. Create a function that dynamically adds a list item to a `ul` each time a button is clicked.
2. Refactor the three validators into one function that takes the field id, error span id, and message.
3. Add a rejection to `factorial` (e.g. num > 15) and show how `catch` handles it in both then-chain and async/await versions.
4. Next-session prep: validate that a password has 8-10 characters with at least one uppercase and one lowercase letter.

46. Final revision checklist

- [ ] I can explain why callbacks exist and why they are called callbacks.
- [ ] I can nest callbacks to make async functions run in order, and I know why it becomes callback hell.
- [ ] I know the three promise states and who decides them.
- [ ] I can write `new Promise((resolve, reject) => ...)` from memory.
- [ ] I can handle promises with `then` / `catch` and pass data through resolve/reject.
- [ ] I can chain promises by returning the next promise from `then`.
- [ ] I can write an async function with `await` and try/catch.
- [ ] I know when to use then/catch and when to use await.
- [ ] I can explain the DOM tree and what DOM manipulation means.
- [ ] I can use getElementById, innerHTML, onclick, style, remove, createElement + appendChild.
- [ ] I can use setAttribute/getAttribute (show/hide password) and know innerHTML vs `.value`.
- [ ] I know why `.value` + `.value` concatenates and how `parseInt` fixes it.
- [ ] I can validate a form using onsubmit, an isValid flag, error spans and a key event.

47. Source-coverage check

Completeness check against the transcript, in lecture order:

Topic discussed in the lecture	Covered in section
Recap of async / setTimeout simulation	1
Why callbacks are needed; why 'callback'	2, 3
Library async functions (sendResponse, readData, writeData examples)	4
Read then send response; read then write; nesting; dependent ops; sync-function question	5
Nested setTimeout sum -> factorial -> square, timings (1.5/4.5/6.5 s)	6
Callback hell and its problems	7
Promise real-life analogy; lifecycle; states; developer decides	8, 9
Promise syntax, executor, resolve/reject (not keywords), pending forever, where to call	10
Even-number promise demo and promise object states	11
then and catch	12
Passing data via resolve/reject	13
Function returning a promise (checkEven, direct return)	14
Promise-based sum/factorial with rejection, dry run, promise returned not value	15
Chaining, wrong nested approach, one catch, square	16
await, async, error, try/catch, async as documentation, scope, then/catch vs await	17-21
DOM definition, tree, CSS/caching Q&A, document object	22, 23
DOM manipulation meaning and capabilities	24, 25
getElementById, innerHTML read/change, onclick, style, remove, createElement/appendChild	26-32
setAttribute / getAttribute, password toggle, bulb task	33
Input .value, calculator, string concatenation, parseInt	34, 35
onsubmit/action, validation, error spans, isValid flag, red CSS class	36
Key events (onkeyup / onkeydown / onchange), live validators, reusable function	37
Next session preview and instructor reminders	38

> **Remember:** Checked against the whole transcript from the opening recap to the closing remarks. Items I added beyond the lecture (extra notes, reconstructed code, interview answers) are labelled as such.